

۳۰ مفهوم کلیدی هوش مصنوعی و مدل‌های زبانی بزرگ

آموزش جامع به زبان فارسی برای مهندسان و علاقه‌مندان به هوش مصنوعی

۱. مدل زبانی بزرگ (LLM – Large Language Model)

LLM یا مدل زبانی بزرگ، یک سیستم هوش مصنوعی است که روی حجم عظیمی از متن آموزش دیده و می‌تواند زبان انسانی را درک کند، تولید کند و با آن استدلال کند.

این مدل‌ها بر پایه معماری **Transformer** ساخته شده‌اند و می‌توانند طیف گسترده‌ای از وظایف زبانی مثل ترجمه، خلاصه‌سازی، پرسش و پاسخ، نوشتن کد و تحلیل متن را انجام دهند.

مثال‌های معروف: GPT-4, Claude, Gemini, LLaMA

چرا مهم است؟ LLMها نحوه تعامل انسان با کامپیوتر را متحول کرده‌اند. به جای نوشتن دستورات دقیق، می‌توانیم به زبان طبیعی با ماشین صحبت کنیم.

۲. توکن (Token)

Token کوچک‌ترین واحد پردازش متن در یک مدل زبانی است. یک توکن ممکن است: - یک کلمه کامل باشد: **سلام** - بخشی از یک کلمه باشد: **un** در **unlikely** - یک علامت نگارشی باشد: **.** یا **!** - یک فاصله باشد

چرا توکن مهم است؟ مدل‌ها متن را به صورت توکن پردازش می‌کنند، نه حرف به حرف یا کلمه به کلمه. هزینه استفاده از APIهای هوش مصنوعی بر اساس تعداد توکن محاسبه می‌شود.

قانون سرانگشتی: در زبان انگلیسی، هر توکن تقریباً ۴ کاراکتر یا ۰.۷۵ کلمه است.

```
"Hello, how are you?" → ["Hello", ",", " how", " are", " you", "?"]
```

توکن 6

۳. توکن‌سازی (Tokenization)

Tokenization فرآیند تبدیل متن خام به دنباله‌ای از توکن‌ها است که مدل می‌تواند پردازش کند.

الگوریتم‌های رایج توکن‌سازی:

روش	توضیح
BPE (Byte Pair Encoding)	رایج‌ترین روش، استفاده شده در GPT
WordPiece	استفاده شده در BERT
SentencePiece	مستقل از زبان، مناسب برای چندزبانگی

مثال BPE:

```
"tokenization" → ["token", "ization"]  
"unhappiness" → ["un", "happiness"]
```

چالش فارسی: زبان فارسی به دلیل ماهیت متصل کلمات، توکن‌سازی دشوارتری دارد و اغلب به توکن‌های بیشتری نسبت به انگلیسی نیاز دارد.

۴. جاسازی‌ها (Embeddings)

Embeddings نمایش عددی (برداری) کلمات، جملات یا مفاهیم در یک فضای ریاضی چندبعدی هستند.

ایده اصلی: مفاهیم مشابه در فضای برداری نزدیک به هم قرار می‌گیرند.

```
vec("پادشاه") - vec("مرد") + vec("زن") ≈ vec("ملکه")
```

ابعاد Embedding: مدل‌های مختلف ابعاد متفاوتی دارند: - مدل‌های کوچک: ۳۸۴ بعد - مدل‌های متوسط: ۷۶۸ بعد - مدل‌های بزرگ: ۱۵۳۶ تا ۳۰۷۲ بعد

کاربردها: - جستجوی معنایی (Semantic Search) - توصیه‌گر (Recommendation System) - تشخیص موضوع متن (Topic Classification) - RAG (که بعداً توضیح می‌دهیم)

۵. فضای پنهان (Latent Space)

Latent Space فضای ریاضی چندبعدی است که مدل در آن مفاهیم را نمایش می‌دهد. این فضا «پنهان» است چون برای انسان قابل تفسیر مستقیم نیست.

تصویرسازی ساده: تصور کنید تمام مفاهیم زبان انسانی را در یک نقشه چندبعدی رسم کنید: - کلمات مرتبط با «غذا» یک خوشه تشکیل می‌دهند - کلمات مرتبط با «ورزش» خوشه دیگری دارند - کلمات با بار احساسی مثبت در یک ناحیه قرار می‌گیرند

چرا مهم است؟ مدل‌ها با عبور اطلاعات از لایه‌های مختلف، مفاهیم را در این فضا رمزگذاری و رمزگشایی می‌کنند. درک Latent Space کمک می‌کند بفهمیم مدل چطور «فکر می‌کند».

۶. پارامترها (Parameters)

Parameters اعداد (وزن‌ها) داخلی مدل هستند که در طول آموزش تنظیم می‌شوند. اندازه مدل معمولاً با تعداد پارامترها بیان می‌شود.

مقایسه اندازه مدل‌ها:

مدل	پارامترها
GPT-2	۱.۵ میلیارد
LLaMA 7B	۷ میلیارد
GPT-3	۱۷۵ میلیارد
GPT-4 (تخمین)	۱+ تریلیون

رابطه پارامتر و توانایی: پارامترهای بیشتر → ظرفیت یادگیری بیشتر → نیاز به داده و محاسبات بیشتر

نوع پارامترها: - **Weights (وزن‌ها):** قدرت ارتباط بین نورون‌ها - **Biases (بایاس‌ها):** مقدار پایه هر نورون

۷. پیش‌آموزش (Pre-training)

Pre-training مرحله اول آموزش مدل است که در آن مدل روی مقادیر عظیمی از متن اینترنتی، کتاب‌ها و مقالات آموزش می‌بیند تا زبان را به صورت کلی یاد بگیرد.

فرآیند: 1. جمع‌آوری صدها گیگابایت تا چند ترابایت متن 2. تبدیل به توکن 3. آموزش مدل برای پیش‌بینی توکن بعدی (Next Token Prediction)

هدف پیش‌آموزش: مدل یاد می‌گیرد: - دستور زبان - واقعیت‌های دنیا - استدلال پایه - ساختار متن‌های مختلف

هزینه پیش‌آموزش: آموزش GPT-4 تخمیناً بیش از ۱۰۰ میلیون دلار هزینه داشته است.

۸. مدل پایه (Base Model)

Base Model خروجی مرحله پیش‌آموزش است؛ مدلی که فقط یاد گرفته متن را ادامه دهد و هنوز برای مکالمه یا دستوردهی بهینه نشده.

رفتار Base Model: اگر بپرسید: «پایتخت ایران چیست؟»

Base Model ممکن است پاسخ دهد:

پایتخت ایران چیست؟
. پایتخت فرانسه پاریس است
. پایتخت آلمان برلین است
...

چون یاد گرفته الگوهای متن را ادامه دهد، نه اینکه سوال را جواب دهد!

چرا مهم است؟ Base Model سنگ‌بنای تمام مدل‌های تخصصی‌تر است. می‌توان آن را برای وظایف مختلف Fine-tune کرد.

۹. مدل دستوری (Instruct Model)

Instruct Model نسخه Fine-tune شده Base Model است که یاد گرفته دستورالعمل‌های انسان را دنبال کند و به شکل مفید پاسخ دهد.

تفاوت با Base Model:

ویژگی	Base Model	Instruct Model
هدف	ادامه متن	پاسخ به دستور
رفتار	پیش‌بینی	همکاری
ایمنی	پایین	بالا

اگر همان سوال را از Instruct Model بپرسید:

پایتخت ایران چیست؟
پایتخت ایران تهران است →

مثال‌های Instruct Model: ChatGPT، Claude، Gemini

۱۰. تنظیم دقیق (Fine-Tuning)

Fine-Tuning فرآیند آموزش مجدد یک مدل از پیش آموزش‌دیده روی یک دیتاست کوچک‌تر و تخصصی‌تر برای انجام وظیفه خاص است.

انواع Fine-Tuning:

Full Fine-Tuning	→ تمام پارامترها آپدیت میشوند (گران‌قیمت)
LoRA	→ فقط ماتریس‌های کوچک اضافه میشوند (سبک‌تر)
QLoRA	→ با کوانتیزه‌سازی (خیلی سبک‌تر) LoRA
PEFT	→ چتر تمام روش‌های بهینه

کاربرد: - مدل پزشکی: Fine-tune روی مقالات پزشکی - مدل حقوقی: Fine-tune روی متون قانونی - مدل کدنویسی: Fine-tune روی کد GitHub

قانون طلایی: Fine-Tuning سبک کار مدل را تغییر می‌دهد، نه دانش پایه آن را.

۱۱. هم‌راستایی (Alignment)

Alignment فرآیند هم‌راستا کردن رفتار مدل با ارزش‌ها، اهداف و انتظارات انسان است.

مشکل بدون Alignment: مدل ممکن است: - اطلاعات خطرناک بدهد - دروغ بگوید - تبعیض‌آمیز رفتار کند - اهداف مضر را دنبال کند

روش‌های Alignment:

۱. RLHF (در مفهوم بعد توضیح داده می‌شود)

۲. Constitutional AI: مجموعه قوانین اخلاقی برای مدل

۳. Direct Preference Optimization (DPO): یادگیری مستقیم از ترجیحات

چالش Alignment: این یکی از مهم‌ترین مسائل پژوهشی AI است. اطمینان از اینکه مدل‌های قدرتمند واقعاً برای انسان مفید و ایمن باشند.

۱۲. یادگیری تقویتی از بازخورد انسانی (RLHF – Reinforcement Learning from Human Feedback)

RLHF روشی است که مدل را با استفاده از بازخورد مستقیم انسان بهبود می‌دهد.

مراحل RLHF:

اولیه با نمونه‌های آموزشی Fine-tuning: مرحله ۱
↓
(Reward Model) مرحله ۲: آموزش مدل پاداش
انسان‌ها چندین پاسخ را رتبه‌بندی می‌کنند
↓
(Proximal Policy Optimization) PPO مرحله ۳: بهینه‌سازی با
مدل یاد می‌گیرد پاسخ‌هایی تولید کند که امتیاز بالا می‌گیرند

نتیجه: مدل یاد می‌گیرد پاسخ‌هایی بدهد که انسان‌ها ترجیح می‌دهند: مفیدتر، صادق‌تر و ایمن‌تر.

مشکلات RLHF: - گران‌قیمت (نیاز به برچسب‌زنی انسانی) - ممکن است مدل یاد بگیرد پاسخ «خوش‌آیند» بدهد نه «درست»

۱۳. پرامپت (Prompt)

Prompt ورودی متنی است که شما به مدل می‌دهید. این ورودی می‌تواند سوال، دستور، یا هر متنی باشد که مدل باید به آن پاسخ دهد.

اجزای یک Prompt خوب:

[قالب خروجی] + [مثال (اختیاری)] + [دستور] + [زمینه]

مثال ضعیف:

خلاصه بده

مثال قوی:

متن زیر یک مقاله علمی است. یک خلاصه ۳ جمله‌ای بنویس که

- ایده اصلی را توضیح دهد
- روش تحقیق را ذکر کند
- نتیجه مهم را بیان کند

متن: [...]

Prompt Engineering: هنر نوشتن Prompt‌های موثر برای گرفتن بهترین نتیجه از مدل. این یک مهارت عملی مهم است.

۱۴. پرامپت سیستم (System Prompt)

System Prompt دستورالعمل‌های پنهانی هستند که قبل از مکالمه کاربر به مدل داده می‌شوند و رفتار کلی مدل را تنظیم می‌کنند.

کاربردها: - تعریف شخصیت مدل - محدود کردن موضوعات مجاز - تعیین زبان پاسخ - تعریف قوانین و محدودیت‌ها

مثال:

System Prompt:

تو یک دستیار پزشکی حرفه‌ای هستی. فقط به سوالات پزشکی پاسخ می‌دهی. همیشه توصیه می‌کنی برای تشخیص دقیق به "پزشک مراجعه شود. پاسخ‌هایت باید علمی و دقیق باشند"

نکته امنیتی: Prompt Injection حمله‌ای است که در آن کاربر سعی می‌کند با دستکاری ورودی، System Prompt را دور بزند.

۱۵. پرامپت کاربر (User Prompt)

User Prompt همان چیزی است که کاربر واقعی تایپ می‌کند و به مدل می‌فرستد.

تفاوت System و User Prompt:

System Prompt (پنهان از کاربر)
"هستی X تو یک دستیار پشتیبانی شرکت"
User Prompt (از کاربر)
"چطور می‌توانم حسابم را حذف کنم؟"
Assistant Response
"... برای حذف حساب مراحل زیر را"

ساختار مکالمه در API:

```
[
  {"role": "system", "content": "..."},
  {"role": "user", "content": "..."},
  {"role": "assistant", "content": "..."},
  {"role": "user", "content": "..."}
]
```

۱۶. پنجره متنی (Context Window)

Context Window حداکثر مقدار متنی است که مدل می‌تواند در یک بار پردازش کند؛ شامل تاریخچه مکالمه، System Prompt و ورودی جدید.

اندازه Context Window مدل‌های مختلف:

مدل	Context Window
GPT-3.5	۱۶K توکن
GPT-4	۱۲۸K توکن
Claude 3	۲۰۰K توکن
Gemini 1.5 Pro	۱ میلیون توکن

مشکلات Context Window کوچک: - فراموش کردن اول مکالمه - ناتوانی در تحلیل اسناد طولانی - محدودیت در پروژه‌های کد بزرگ

Lost in the Middle: تحقیقات نشان می‌دهد مدل‌ها اطلاعات وسط Context Window را بدتر از ابتدا و انتها یادآوری می‌کنند.

۱۷. یادگیری بدون نمونه (Zero-Shot Learning)

Zero-Shot Learning توانایی مدل در انجام وظیفه‌ای است که هیچ مثالی از آن در Prompt وجود ندارد.

مثال:

Prompt: "The sky is blue": جمله زیر را به فارسی ترجمه کن"

مدل بدون دیدن هیچ نمونه ترجمه‌ای، ترجمه می‌کند →
"آسمان آبی است"

چرا کار می‌کند؟ مدل در طول Pre-training این دانش را کسب کرده و می‌تواند آن را به وظایف جدید تعمیم دهد.
محدودیت: برای وظایف بسیار تخصصی یا غیرمعمول، Zero-Shot ممکن است کافی نباشد.

۱۸. یادگیری با چند نمونه (Few-Shot Learning)

Few-Shot Learning وقتی است که چند مثال (معمولاً ۱ تا ۱۰) در Prompt قرار می‌دهید تا مدل الگو را یاد بگیرد.

مثال:

ورودی: "خوشحال" → احساس: مثبت
ورودی: "غمگین" → احساس: منفی
ورودی: "بی‌تفاوت" → احساس: خنثی
؟: ورودی: "هیجان‌زده" → احساس

مدل پاسخ می‌دهد: مثبت

انواع: **One-Shot**: یک مثال - **Few-Shot**: ۲ تا ۱۰ مثال - **Many-Shot**: دهها تا صدها مثال (در Context Window بزرگ)

چه وقت استفاده کنیم؟ وقتی قالب خروجی مشخص دارید یا وظیفه غیرمعمول است.

۱۹. زنجیره فکر (Chain-of-Thought – CoT)

Chain-of-Thought تکنیکی است که مدل را وادار می‌کند مرحله به مرحله استدلال کند قبل از رسیدن به جواب نهایی.

بدون CoT:

سوال: اگر علی ۵ سیب داشته باشد و ۳ تا به مریم بدهد، چند تا دارد؟
پاسخ: ۳ - (غلط!)

با CoT:

سوال: اگر علی ۵ سیب داشته باشد و ۳ تا به مریم بدهد، چند تا دارد؟
:بذار قدم به قدم فکر کنیم
علی در ابتدا ۵ سیب دارد -
مریم ۳ سیب می‌گیرد -
 $5 - 3 = 2$ -
✓ پاسخ: علی ۲ سیب دارد

انواع CoT: - **Zero-Shot CoT**: فقط "بذار قدم به قدم فکر کنیم" اضافه می‌کنیم - **Few-Shot CoT**: مثال‌هایی با استدلال کامل می‌دهیم

چرا کار می‌کند؟ استدلال مرحله‌ای خطاها را کاهش می‌دهد چون مدل نمی‌تواند پاسخ را حدس بزند؛ باید مسیر منطقی را طی کند.

۲۰. استنتاج (Inference)

Inference فرآیند استفاده از مدل آموزش‌دیده برای تولید پاسخ است. به عبارت دیگر، اجرای مدل در حالت عملیاتی (نه آموزش).

تفاوت **Training vs Inference**:

Inference (استنتاج)	Training (آموزش)	
استفاده از وزن‌ها	یادگیری وزن‌ها	هدف
کمتر	بسیار زیاد	منابع
میلی‌ثانیه	هفته‌ها	زمان
ثابتند	تغییر می‌کنند	وزن‌ها

بهینه‌سازی Inference - Quantization: کاهش دقت عددی (از float32 به int8) - **Batching:** پردازش همزمان چند درخواست - **Caching:** ذخیره KV-Cache برای مکالمات طولانی

۲۱. تأخیر (Latency)

Latency زمانی است که از ارسال درخواست تا دریافت پاسخ طول می‌کشد.

انواع **Latency** در **LLM**:

Time to First Token (TTFT)

- زمان از ارسال سوال تا اولین کلمه پاسخ
- مهم برای تجربه کاربری

Tokens Per Second (TPS)

- سرعت تولید توکن‌ها
- مهم برای پاسخ‌های طولانی

عوامل موثر بر Latency: - اندازه مدل (مدل بزرگتر = کندتر) - طول Context Window - سخت‌افزار سرور - فشار بار (Load) روی سرویس

بهینه‌سازی: - استفاده از مدل‌های کوچک‌تر برای کارهای ساده - Streaming: نمایش پاسخ در حین تولید (مثل ChatGPT) - Edge deployment: اجرای مدل نزدیک کاربر

۲۲. دما (Temperature)

Temperature پارامتری است که میزان تصادفی بودن یا خلاقیت پاسخ مدل را کنترل می‌کند.

محدوده: معمولاً بین ۰ تا ۲

Temperature = 0.0

- قطعی‌ترین پاسخ، همیشه محتمل‌ترین توکن انتخاب می‌شود
- مناسب: کد، محاسبات، پاسخ‌های دقیق

Temperature = 0.7

- تعادل بین دقت و خلاقیت
- مناسب: مکالمه عمومی، پاسخ‌دهی

Temperature = 1.5+

- خلاقانه اما گاهی نامرتبط
- مناسب: شعر، داستان، ایده‌پردازی

مثال: با $Temperature = 0$ ، پاسخ به "سلام" همیشه یکسان است. با $Temperature = 1.5$ ، هر بار پاسخ متفاوتی می‌گیرید.

پارامترهای مرتبط: - **Top-P (nucleus sampling):** فقط از محتمل‌ترین توکن‌ها انتخاب کن - **Top-K:** از K توکن برتر انتخاب کن

۲۳. توهم (Hallucination)

Hallucination وقتی است که مدل اطلاعات نادرست، ساختگی یا گمراه‌کننده تولید می‌کند اما با اعتماد به نفس کامل آن را بیان می‌کند.

انواع Hallucination:

نوع	مثال
واقعی	تاریخ اشتباه، آمار غلط
منبع	ذکر کتاب یا مقاله‌ای که وجود ندارد
هویت	اشتباه در نسبت دادن سخن به شخص
منطقی	نتیجه‌گیری اشتباه از اطلاعات درست

چرا اتفاق می‌افتد؟ مدل‌ها برای تولید متن آموزش دیده‌اند، نه برای «دانستن حقیقت». وقتی اطلاعات ندارند، ممکن است الگوی محتمل را تولید کنند.

راه‌های کاهش: - استفاده از RAG (منابع خارجی) - درخواست استناد به منابع - بررسی دوباره اطلاعات مهم - استفاده از مدل‌های جدیدتر

۲۴. زمین‌گیری (Grounding)

Grounding فرآیند اتصال پاسخ‌های مدل به اطلاعات واقعی، معتبر و قابل تأیید است.

چرا لازم است؟ مدل‌ها بر اساس دانش آموزشی خود پاسخ می‌دهند که ممکن است: - قدیمی باشد (Knowledge Cutoff) - نادرست باشد (Hallucination) - برای زمینه خاص کافی نباشد

روش‌های Grounding:

۱. RAG (Retrieval Augmented Generation)
اتصال به پایگاه داده یا اسناد -
۲. Web Search
جستجو در اینترنت قبل از پاسخ -
۳. Tool Use / Function Calling
(API, calculator) استفاده از ابزارهای خارجی -
۴. Citation
الزام به ذکر منبع برای هر ادعا -

مثال عملی: بدون Grounding: «جمعیت تهران X میلیون است» (شاید اشتباه) با Grounding: مدل ابتدا آمارنامه ۱۴۰۳ را می‌خواند، سپس پاسخ می‌دهد.

۲۵. تولید با بازیابی (RAG – Retrieval Augmented Generation)

RAG روشی است که مدل قبل از تولید پاسخ، اطلاعات مرتبط را از یک پایگاه داده دانش بازیابی می‌کند و آن را در پاسخ استفاده می‌کند.

معماری RAG:

سوال کاربر

↓

Embedding تبدیل به

↓

(Vector Database) جستجوی شباهت

↓

بازیابی اسناد مرتبط

↓

Prompt → ترکیب سوال + اسناد

↓

مدل پاسخ میدهد

↓

پاسخ مستند و دقیق

مزایا: - اطلاعات به روز (نیازی به Re-training نیست) - قابل استناد (می‌توان منابع را نشان داد) - ارزان‌تر از Fine-Tuning - کنترل دقیق روی محتوا

کاربردها: - چت‌بات مستندات شرکت - دستیار پزشکی با پایگاه داده بیمار - جستجوی هوشمند در اسناد حقوقی

۲۶. گردش کار (Workflow)

Workflow دنباله‌ای از مراحل از پیش تعریف‌شده است که LLM در کنار سایر ابزارها برای انجام وظایف پیچیده طی می‌کند.

تفاوت با Agent: در Workflow مسیر مشخص است؛ در Agent مدل خودش مسیر را انتخاب می‌کند.

مثال Workflow ساده:

مقاله URL ورودی: یک

↓ (Tool Call) مرحله ۱: دریافت محتوای صفحه

↓ (LLM) مرحله ۲: خلاصه‌سازی

↓ (LLM) مرحله ۳: استخراج نکات کلیدی

↓ (Tool Call) مرحله ۴: ذخیره در دیتابیس

خروجی: خلاصه ذخیره‌شده

ابزارهای ساخت Workflow: LangChain - LlamaIndex - Haystack - Prefect + LLM

۲۷. عامل (Agent)

Agent یک سیستم هوش مصنوعی است که می‌تواند مستقل تصمیم بگیرد، ابزارها را انتخاب کند، و برای رسیدن به هدف اقدام کند.

اجزای یک Agent:



چرخه **ReAct (Reason + Act)**:

... → فکر کن → اقدام کن → مشاهده کن → فکر کن

مثال عملی:

"کاربر: آخرین اخبار بیت‌کوین را بررسی کن و خلاصه بده"

Agent:

1. فکر می‌کند: باید اخبار را جستجو کنم
2. اقدام: `web_search("bitcoin news today")`
3. مشاهده: نتایج جستجو
4. فکر می‌کند: باید خلاصه کنم
5. اقدام: `summarize(نتایج)`
6. پاسخ نهایی به کاربر

انواع Agent: **ReAct Agent**: استدلال + عمل - **Plan-and-Execute**: برنامه‌ریزی کامل سپس اجرا - **Multi-Agent**: چند Agent با هم کار می‌کنند

۲۸. چندوجهی (Multimodality)

Multimodality توانایی مدل در پردازش و تولید انواع مختلف داده فراتر از متن است.

انواع Modal:

ورودی‌ها (Input Modalities):

- متن (Text)
- تصویر (Image)
- صدا (Audio)
- ویدیو (Video)
- اسناد (Documents/PDF)

خروجی‌ها (Output Modalities):

- متن
- تصویر (Image Generation)
- صدا (Text-to-Speech)
- کد

مدل‌های Multimodal معروف:

مدل	قابلیت‌ها
GPT-4V	متن + تصویر
Claude 3	متن + تصویر + PDF
Gemini 1.5	متن + تصویر + صدا + ویدیو
DALL-E 3	متن → تصویر
Whisper	صدا → متن

کاربرد: - تحلیل نمودارهای مالی از تصویر - پاسخ به سوال درباره محتوای ویدیو - تبدیل دستنوشته به متن

۲۹. معیارسنجی (Benchmarks)

Benchmarks آزمون‌های استاندارد هستند که برای مقایسه عملکرد مدل‌های مختلف استفاده می‌شوند.

معیارهای معروف:

معیار	هدف
MMLU	دانش عمومی (57 موضوع)
HumanEval	کدنویسی
GSM8K	ریاضیات مدرسه
MATH	ریاضیات پیشرفته
HellaSwag	استدلال زبانی
TruthfulQA	صداقت و دقت
GPQA	سوالات دکترا

انواع ارزیابی:

Automated Benchmarks

- سریع و ارزان
- داشته باشد Data Leakage ممکن است

Human Evaluation

- دقیقتر و جامعتر
- گران و کند

Arena-Style (Chatbot Arena)

- رأی‌گیری از انسان‌ها بین مدل‌ها
- نزدیک‌ترین به استفاده واقعی

هشدار مهم: بعضی مدل‌ها روی Benchmark ها بهینه‌سازی می‌شوند (**Benchmark Hacking**). عملکرد واقعی را در استفاده روزمره بسنجید.

۳۰. حفاظ‌ها (Guardrails)

Guardrails سیستم‌ها و مکانیزم‌هایی هستند که رفتار مدل را محدود و کنترل می‌کنند تا خروجی ایمن، مناسب و مطابق با قوانین باشد.

انواع Guardrails:

Input Guardrails (فیلتر ورودی)

- شناسایی محتوای مضر در درخواست
- Prompt Injection جلوگیری از
- بررسی انطباق با قوانین

Output Guardrails (فیلتر خروجی)

- بررسی محتوا قبل از ارسال به کاربر
- (PII) حذف اطلاعات حساس
- Hallucination تشخیص

سطوح Guardrails:

سطح	مثال
مدل	RLHF، Constitutional AI
برنامه	فیلتر کلمات، بررسی موضوع
سیستم	Rate limiting، نظارت

ابزارهای معروف: - **NeMo Guardrails** (NVIDIA) - **LlamaGuard** (Meta) - **Prompt shields** (Azure) - **Guardrails AI** (open source)

تعادل مهم: Guardrails بیش از حد → مدل بی‌فایده می‌شود Guardrails ناکافی → مدل خطرناک می‌شود

مرحله ۱: پایه‌ها

LLM → Token → Tokenization → Embeddings → Latent Space

مرحله ۲: آموزش مدل

Parameters → Pre-training → Base Model → Fine-Tuning

مرحله ۳: بهبود رفتار

Alignment → RLHF → Instruct Model

مرحله ۴: تعامل با مدل

Prompt → System Prompt → User Prompt → Context Window

مرحله ۵: تکنیک‌های استفاده

Zero-Shot → Few-Shot → Chain-of-Thought

مرحله ۶: عملیات

Inference → Latency → Temperature

مرحله ۷: چالش‌ها و راه‌حل‌ها

Hallucination → Grounding → RAG

مرحله ۸: سیستم‌های پیشرفته

Workflow → Agent → Multimodality

مرحله ۹: ارزیابی و کنترل

Benchmarks → Guardrails

منابع برای یادگیری بیشتر

• **CS324 Stanford**: دوره LLMs دانشگاه استنفورد

• **fast.ai**: دوره‌های عملی یادگیری ماشین

• **Hugging Face Course**: آموزش رایگان Transformers

• **Andrej Karpathy YouTube**: توضیحات عمیق از مهندس سابق OpenAI

• **LangChain Docs**: ساخت برنامه‌های LLM

این آموزش پوشش دهنده ۳۰ مفهوم کلیدی LLM برای مهندسان AI است.